KCC CODE CLUB · GAME MAKERS

Week 2 Lesson Plan

Events and Movement

Big idea: Code doesn't only run when you click the flag. It runs when *things happen* — and position on screen is just two numbers.

At a glance

New concepts	Events, coordinates, several scripts running at once
New blocks	`when [key] pressed`, `when this sprite clicked`, `change x by`, `change y by`, `set x to`, `set y to`, `glide n secs to x y`, `point in direction`
Class activity	Drive the Car
Project checkpoint	The player moves left and right with the arrow keys
Homework	Up/down or screen wrap; make the player face where it's going

Before the class

- Print the Week 2 handout.
- Have a large coordinate grid ready — whiteboard, or masking tape on the floor if you have space. The floor version is much better and worth the setup.
- Skeleton file `Sky Catcher – end of Week 1` ready to send to anyone who missed class one.

0:00–0:05 — Show & Tell

Two volunteers demo their homework joke on the projector. Applaud everything.

If nobody volunteers — likely in week two — demo one yourself and ask the class to rate the joke out of ten. Next week someone will put their hand up.

0:05–0:15 — Live demo: hat blocks are triggers

Start with the point: every script needs a **hat block** — a trigger that says *when* to run. The green flag is only one of them.

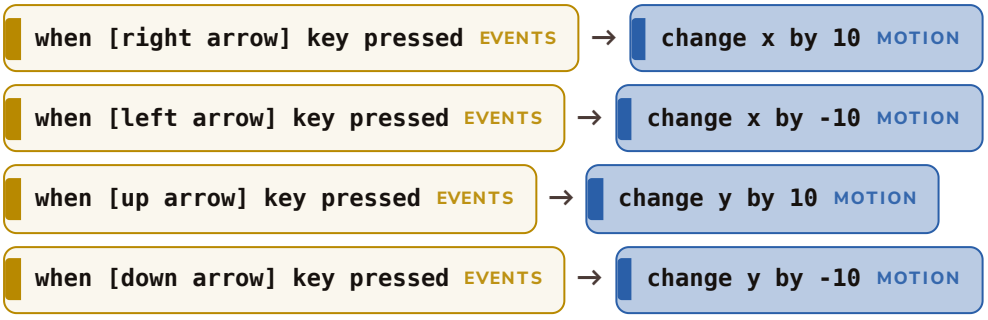
Build live:

1. `when [space] key pressed` → `change x by 10`. Run it by pressing space. No green flag needed. Let that land; several of them will assume everything needs the flag.
2. Add a second script on the same sprite: `when [right arrow] key pressed` → `change x by 10`. Now two scripts, one sprite. Point out that both are live at the same time.
3. `when this sprite clicked` → `say [ouch!]`. Click the sprite. Three triggers now.

4. Show `change x by 10` versus `set x to 10` and make them tell you the difference. *Change* is relative, *set* is absolute. This distinction comes back hard in week five with variables, so it's worth spending a minute on now.
5. Show `glide 1 secs to x: 0 y: 100` — smooth movement, versus the teleport of `set`.
6. **Deliberate mistake:** use `set x to 10` for movement and press the key repeatedly. It only moves once. Be baffled, then work it out with them.

0:15–0:30 — Class activity: Drive the Car

A sprite controlled by all four arrow keys.



Then let them play with the number. Change 10 to 50 — too fast. To 2 — too slow. Finding the number that *feels* right is their first taste of tuning, and they enjoy it more than you'd expect.

If they finish early: bonus challenges. The pen-trail one keeps a fast student busy for fifteen minutes. Treasure Hunt, bonus challenge 2, is the one to give anyone who wants more practice with `when this sprite clicked`.

0:30–0:55 — Project build: the player moves

Goal in one sentence: **"By the end of today you can drive your player left and right along the bottom."**

Blocks on the board, unordered: `when [right arrow] key pressed`, `when [left arrow] key pressed`, `change x by`, `go to x y`.

Let them assemble it. Then two refinements, once most of them have it working:

1. **Stay at the bottom.** Only x should change. If they've added up/down movement, that's fine for now, but the falling objects in week three assume the player lives near the bottom.
2. **Don't escape the stage.** Optional and slightly fiddly. If someone asks how to stop the player leaving the screen, the simple answer is `if x position > 220 then set x to 220`, but `if` isn't taught until week four — so give it as a copy-this-for-now if they push, or defer it honestly: *"we'll be able to do that properly in two weeks."*

Deferring honestly is better than a magic block they don't understand.

Save. Check the name.

0:55–1:00 — Wrap

Save, back up, brief the homework, point at the bonuses.

Homework

1. **Add another dimension.** Either let your player move up and down as well, or make it wrap around — go off the right edge, come back on the left. (`if` isn't taught yet, so wrapping needs `if on edge, bounce` or `set x to -240` on a key. Either is a fine answer.)
2. **Face the right way.** Make your player point in the direction it's moving, using `point in direction`.

Bonus challenges

1. Add WASD keys as well as the arrows, so two people could play.
2. **Treasure Hunt.** Three or four sprites on the stage, one hiding treasure. Each empty one: `when this sprite clicked` → `say [Nothing here!] for 2 seconds`. The winner says `[You found it!]` and does `change size by 50`.
3. Add the Pen extension and make your sprite draw a trail as it moves. Add a key that clears it.
4. Make the sprite move faster when you hold down the shift key. (Harder than it looks — a good one to leave hanging until week four.)

Common bugs this week

Symptom	Cause
Sprite only moves once per key press, then stops	Used `set x to` instead of `change x by`
Sprite moves diagonally when they wanted straight	Both x and y are changing; check the block
Nothing happens on key press	Wrong sprite selected when the script was written
Sprite leaves the stage and won't come back	Add `go to x:0 y:-140` under the green flag
Sprite is on its side or upside down	`point in direction` combined with the sprite's rotation style. Set rotation style to "left-right" in the sprite pane

That last one arrives every year with the `point in direction` homework. The fix is in the sprite info panel, not in the code, which is why it stumps them.

Notes for the second volunteer

Watch for children who've built the right script on the *stage* instead of on a sprite. The stage has no motion blocks, so the palette looks broken to them and they get quietly stuck.

Links

- Catch the bus — Raspberry Pi Foundation, good extension for movement
- Skeleton: `Sky Catcher — end of Week 2` in the club studio